

Ответы к экзаменационным вопросам «ОПИ» 2025/2026

软件工程基础 (OPI) 课程期末考试题指南 (1 - 80 题)

Полный двуязычный справочник (Русский / 中文) — Все 80 вопросов

1. ISO/IEC 12207:2010: Жизненный цикл ПО. Группы процессов ЖЦ.

ISO/IEC 12207:2010: 软件生命周期。生命周期过程组。

Стандарт ISO/IEC 12207 определяет процессы жизненного цикла ПО, разделяя их на 7 основных групп: соглашения, организационные, проекта, технические, реализации, логистики и основные процессы.

ISO/IEC 12207:2010 标准定义了软件生命周期过程，主要划分为 7 个过程组：协议过程、组织性过程、项目过程、技术过程、实现过程、支持过程及运营/维护过程。

2. Модели ЖЦ (последовательная, инкрементная, эволюционная).

生命周期模型（顺序模型、增量模型、演化模型）。

Последовательная модель выполняет фазы строго друг за другом (Waterfall). Инкрементная разделяет систему на подсистемы и создает их поэтапно. Эволюционная развивается на основе постоянной обратной связи с пользователем.

顺序模型按严格阶段依次进行（如瀑布模型）；增量模型将系统拆分为子系统并分批交付；演化模型基于持续的用户反馈迭代演进。

3. Водопадная (каскадная) модель.

瀑布模型（级联模型）。

Классическая линейная модель (Анализ -> Проектирование -> Кодинг -> Тестирование -> Внедрение). Каждая фаза завершается документированием. Недостаток: сложность внесения изменений на поздних этапах.

经典的线性顺序模型（需求分析 -> 设计 -> 编码 -> 测试 -> 部署）。每个阶段均需输出文档。缺点：后期修改变更成本高、风险高。

4. Методология Ройса.

Royce 瀑布模型改进方法论。

Уинстон Ройс в 1970 г. предложил каскадную модель с обратными связями (итерациями) между соседними этапами и обязательным прототипированием перед полной реализацией.

Winston Royce 于1970年提出带反馈循环的瀑布模型，强调相邻阶段间的迭代以及在全面实现前先进行原型设计。

5. Традиционная V-chart model J.Munson, B.Boehm.

传统 V 模型 (J.Munson, B.Boehm)。

V-модель связывает каждый этап разработки с соответствующей фазой тестирования (Требования <-> Приемочные тесты, Архитектура <-> Интеграционные тесты, Модули <-> Юнит-тесты).

V模型将开发过程与测试过程一一对应（需求分析对应验收测试、架构设计对应集成测试、模块编码对应单元测试）。

6. Многопроходная модель (Incremental model).

多趟/增量模型 (Incremental Model)。

Разработка разбивается на ряд выпусков (инкрементов). Каждый инкремент добавляет новую функциональность. Первыми создаются базовые модули, затем дополнительные.

将项目拆分为多个增量发布版。每个增量增加部分功能。首先构建核心功能模块，后续增量再逐步扩充辅助功能。

7. Модель прототипирования (80-е).

原型设计模型 (80年代) 。

Создание макета (прототипа) для уточнения требований заказчика. После утверждения прототипа начинается основная разработка. Снижает риски недопонимания.

通过快速制作软件原型（静/动态样机）来明确和确认用户需求。在原型确认后再启动系统研发，能大幅降低需求偏差风险。

8. RAD методология.

RAD 快速应用开发方法论。

Rapid Application Development — сочетание итеративной разработки, прототипирования, использования CASE-средств и визуального программирования с короткими циклами (60-90 дней).

快速应用开发：结合迭代开发、原型制作、可视化编程工具及组件复用，强调短周期（60-90天内）快速交付可用软件。

9. Спиральная модель.

螺旋模型 (Spiral Model, B.Boehm)。

Предложена Б. Боэмом. Сочетает итеративность и водопадный подход с акцентом на анализ рисков на каждом витке спирали (Планирование -> Анализ рисков -> Конструирование -> Оценка).

由 Barry Boehm 提出，结合了瀑布模型和增量模型的优点，核心特点是在螺旋的每一个循环阶段中都引入了严格的风险分析。

10. UML Диаграммы: Структурные и поведенческие. Примеры диаграмм.

UML 绘图：结构图与行为图及示例。

Структурные (статические): Диаграммы классов, компонентов, размещения. Поведенческие (динамические): Use-case, диаграммы состояний, последовательности, активности.

结构图（静态）：类图、组件图、部署图等；行为图（动态）：用例图、状态机图、时序图、活动图等。

11. UML: Use-case модель. Элементы диаграммы, пример диаграммы.

UML:用例模型 (Use-case Model)、图要素及示例。

Показывает функциональность системы с точки зрения актеров (пользователей/систем).

Элементы: Актер (Actor), Прецедент (Use Case), Связи (Include, Extend, Association).

从参与者（用户或外部系统）视角展现系统功能。关键要素：Actor（参与者）、Use Case（用例）、关系（包含 Include、扩展 Extend、关联 Association）。

12. UML: Диаграмма классов. Элементы диаграммы, пример диаграммы.

UML: 类图 (Class Diagram)、图要素及示例。

Отображает статическую структуру системы. Элементы: Класс (имя, атрибуты, методы),

Интерфейс, Связи (Наследование/Generalization, Агрегация, Композиция, Зависимость).

描述系统的静态结构。元素包括：类（类名、属性、方法）、接口、以及类间关系（继承/泛化、聚合、组合、关联、依赖）。

13. UML: Диаграмма состояний. Элементы диаграммы, пример диаграммы.

UML: 状态图 (State Diagram)、图要素及示例。

Моделирует жизненный цикл одного объекта. Элементы: Начальное/конечное состояние,

Состояние (State), Переход (Transition), Событие (Event), Действие (Guard/Action).

用于建模单个对象的生命周期及其状态迁移。元素包括：初始/终止状态、状态节点、状态转移线、触发事件及条件约束。

14. UML: Диаграмма размещения. Элементы диаграммы, пример диаграммы.

UML: 部署图 (Deployment Diagram)、图要素及示例。

Отображает физическую конфигурацию узлов обработки и компонентов. Элементы: Узел (Node), Артефакт (Artifact), Соединение (Communication Path).

展现系统硬件物理节点及在其上部署的软件构件。元素包括：Node（计算节点）、Artifact（构件/部署文件）、Communication Path（通信连接）。

15. *UP методологии (90-e). RUP: основы процесса.

UP 方法论（90年代）与 RUP 核心基础。

Rational Unified Process — управляемый прецедентами, архитекторо-центричный, итеративный и инкрементный процесс. Делится на 4 фазы и 9 дисциплин.

统一软件开发过程（RUP）：用例驱动、以架构为中心、迭代与增量开发。划分为 4 个阶段和 9 个核心学科/ workflow。

16. RUP: Фаза «Начало».

RUP: 初始阶段 (Inception Phase)。

Определяются цели проекта, границы системы, основные прецеденты и оценки затрат/рисков.

Формируется бизнес-кейс (Business Case).

确定项目目标、系统边界、关键用例、业务可行性分析及初步预算与风险评估。

17. RUP: Фаза «Проектирование».

RUP: 精化阶段 (Elaboration Phase) 。

Анализ предметной области, архитектурный проект, базовый каркас системы (Architectural Baseline), уточнение большинства требований и рисков.

详细分析业务领域、设计系统软件架构、构建可运行的架构基线，精化绝大部分需求并规避主要风险。

18. RUP: Фаза «Построение».

RUP: 构建阶段 (Construction Phase) 。

Полная разработка системы (кодинг, оптимизация, тестирование компонентов). Создание готового продукта для альфа/бета тестирования.

完成所有系统特性的开发与测试（编写代码、单元测试与集成测试），最终生成包含完整功能的软件产物。

19. RUP: Фаза «Внедрение».

RUP: 交付阶段 (Transition Phase) 。

Развертывание системы у заказчика, обучение пользователей, бета-тестирование, исправление выявленных дефектов и сдача в эксплуатацию.

将软件部署至最终用户环境、进行用户培训、进行 Alpha/Beta 试运行与缺陷修复，完成验收与正式交付。

20. Манифест Agile (2001).

敏捷宣言 (Agile Manifesto, 2001) 。

4 ценности: Люди и взаимодействие > Процессы и инструменты; Работающий продукт > Документация; Сотрудничество с клиентом > Согласование условий; Готовность к изменениям > Следование плану.

4项核心价值观：个体与交互胜过过程与工具；可工作的软件胜过面面俱到的文档；客户协作胜过合同谈判；响应变化胜过遵循计划。

21. Scrum.

Scrum 敏捷框架。

Итеративный Agile-фреймворк. Роли: Product Owner, Scrum Master, Developers. Артефакты: Product Backlog, Sprint Backlog, Increment. События: Sprint, Planning, Daily, Review, Retro.

敏捷开发框架。三大角色：PO（产品负责人）、SM（敏捷教练）、开发团队；三大工件：产品 Backlog、迭代 Backlog、增量；四大仪式：Sprint 计划会、站会、评审会、复盘会。

22. Disciplined Agile 2.X (2013).

Disciplined Agile 2.X (2013) 框架。

DA 2.X — гибридная методология от Scott Ambler, сочетающая Agile, Lean, Scrum и RUP.

Предоставляет руководства по выбору жизненного цикла под контекст предприятия.

由 Scott Ambler 提出的混合型敏捷框架，整合了 Agile、Lean、Scrum 和 RUP 的精髓，提供适合企业特定上下文的敏捷模式指导。

23. Требования. Иерархия требований. Примеры формулировок на разных уровнях.

需求、需求层次结构及各层示例。

Иерархия: 1) Бизнес-требования (цели бизнеса), 2) Пользовательские требования (Use Cases), 3) Функциональные/Нефункциональные требования, 4) Системные требования.

需求分层: 1) 业务需求 (商业目标); 2) 用户需求 (Use-case/用户视角); 3) 功能/非功能需求 (细节规则与质量); 4) 系统与技术需求。

24. Свойства и типы требований (FURPS+).

需求的属性与 FURPS+ 分类模型。

FURPS+: Functionality (Функциональность), Usability (Удобство), Reliability (Надежность), Performance (Производительность), Supportability (Поддерживаемость) + (+) Ограничения (Design, Implementation, Interface, Physical).

FURPS+ 模型: 功能性、易用性、可靠性、性能、可维护性; “+” 代表设计/实现/接口/物理等约束条件。

25. Формулирование требований. Функциональные требования.

需求表达与功能性需求。

Определяют поведение системы (что система должна делать). Формулируются в формате 'Система должна [действие] при [условии]'. Должны быть точными, однозначными и проверяемыми.

定义系统“应该做什么”的具体行为与逻辑。标准句式: “系统在[条件]下应当[动作]”。要求具备准确性、无二义性及可测试性。

26. Требования к удобству использования и надежности.

易用性 (Usability) 与可靠性 (Reliability) 需求。

Usability: время обучения, доступность UI, эргономика. Reliability: среднее время наработки на отказ (MTBF), время восстановления (MTTR), доступность (Availability %).

易用性: 学习成本、UI友好度、操作效率; 可靠性: 平均无故障时间 (MTBF)、故障修复时间 (MTTR)、系统可用性百分比等。

27. Требования к производительности и поддерживаемости.

性能 (Performance) 与可维护性 (Supportability) 需求。

Performance: время отклика, пропускная способность, потребление ресурсов. Supportability: расширяемость, тестируемость, удобство сопровождения и обновления.

性能: 响应时间、并发吞吐量 (TPS)、资源消耗率; 可维护性: 模块扩展性、可测试性、可移植性及配置升级难易度。

28. Атрибуты требований.

需求的属性 (Attributes)。

Каждое требование имеет атрибуты: Уникальный ID, Приоритет (MoSCoW), Статус (Draft/Approved), Автор, Источник, Прослеживаемость (Traceability).

需求包含的扩展信息：唯一标识符 ID、优先级（MoSCoW 分级）、当前状态、创建者、需求来源、可追溯性标记 (Traceability)。

29. Описание прецедента.

用例说明文档 (Use Case Description)。

Детальный документ прецедента: Название, Актеры, Предусловия, Постусловия, Основной поток (Main Flow), Альтернативные/Исключительные потоки.

用例详细说明书结构：用例名称、参与者、前置条件、后置条件、主成功场景流程（Main Flow）、扩展/异常分支流程。

30. Риски. Типы Рисков.

风险及其分类。

Риск — потенциальное негативное событие. Типы: Проектные (сроки, бюджет), Технические (технологии, архитектура), Бизнес-риски (рынок, заказчик).

风险是未发生的潜在负面事件。分类：项目风险（进度、预算、人员）、技术风险（架构复杂度、技术壁垒）、业务/商业风险。

31. Управления рисками. Деятельности, связанные с оценкой.

风险管理：风险评估相关活动。

Оценка рисков включает: Идентификация рисков, Анализ (качественный — вероятность и ущерб; количественный — финансовая оценка), Приоритезация.

风险评估活动：1) 风险识别（找出潜隐风险）；2) 风险分析（定性评估概率与影响，定量评估损失值）；3) 风险优先级排序。

32. Управления рисками. Деятельности, связанные контролем и управлением.

风险管理：风险控制与监控活动。

Планирование реагирования: Уклонение (Avoidance), Передача (Transfer), Смягчение (Mitigation), Принятие (Acceptance). Мониторинг и регулярный пересмотр.

风险应对规划：回避、转移、减轻（降低发生概率/影响）、接受。持续对风险状态进行监控与动态调整。

33. Изменение. Общая модель управления изменениями.

变更与通用变更管理模型。

Процесс обработки запросов на изменение (RFC): Запрос -> Анализ влияния (Impact Analysis) -> Оценка стоимости/сроков -> Одобрение (CCB) -> Реализация -> Проверка.

变更控制流程 (RFC)：提交变更请求 -> 影响分析 -> 评估成本与工期 -> 变更控制委员会 (CCB) 审批 -> 实施变更 -> 验证与发布。

34. Системы контроля версий. Одновременная модификация файлов.

版本控制系统与文件并发修改解决机制。

VCS отслеживают историю файлов. Подходы к параллельной работе: 1) Lock-Modify-Unlock (блокировка), 2) Copy-Modify-Merge (копирование-модификация-слияние, как в SVN/Git).

VCS 用于追踪文件演进历史。并发修改解决方案：1) 独占锁模式 (Lock-Modify-Unlock) ; 2) 复制-修改-合并模式 (Copy-Modify-Merge, 如 SVN/Git) 。

35. Subversion. Архитектура системы и репозиторий.

Subversion (SVN): 系统架构与集中式仓库。

SVN — централизованная VCS (клиент-сервер). Репозиторий хранит деревья каталогов и версионирует весь проект как единую файловую систему (ревизии N).

SVN 是典型的集中式版本控制系统 (客户端-服务器架构)。服务器中心仓库保存所有文件历史，每次提交均增加全局全局递增版本号 (Revision)。

36. Subversion: Основной цикл разработчика. Команды.

Subversion (SVN): 开发者基本工作循环与指令。

Цикл: svn checkout (получить), svn update (обновить), редактирование, svn status / diff (проверка), svn add (добавить), svn commit (фиксация).

典型循环: checkout 检出 -> update 更新本地 -> 修改代码 -> status / diff 检查变更 -> add 添加新文件 -> commit 提交到服务器。

37. Subversion: Конфликты. Слияние изменений.

Subversion (SVN): 冲突处理与代码合并。

Конфликт возникает при правке одной строки разными разработчиками. SVN создает файлы .mine, .rOLD, .rNEW. Разрешение: ручное редактирование и svn resolve.

当多人在同一位置修改时产生冲突，SVN 会生成 .mine、.rOLD、.rNEW 标记文件。需人工编辑保留正确代码后执行 svn resolve。

38. GIT: Архитектура и команды.

Git: 分布式架构与常用命令。

Git — распределенная VCS. Три области: Working Directory, Staging Area (Index), Repository (.git).

Команды: git init, add, commit, status, log, push, pull, checkout.

Git 是分布式版本控制系统。三大工作区：工作区、暂存区 (Index) 和本地仓库。常用命令：init, add, commit, status, log, push, pull, checkout 等。

39. GIT: Организация ветвей репозитория.

Git: 仓库分支管理策略。

Ветвление в Git дешевое (указатель на коммит). Модели: Feature Branch Workflow (ветки под фичи), Trunk-Based Development (короткоживущие ветки).

Git 分支轻量（本质是指向提交的指针）。常见策略：功能分支流（Feature Branching）、主干开发模式（Trunk-Based Development）。

40. GIT: Плагин git-flow.

Git: Git-Flow 分支 workflow 模型。

Модель Git-flow определяет постоянные ветки (master, develop) и временные (feature/*, release/*, hotfix/*) для строгой организации релизного цикла.

Git-Flow 明确划分了长久分支（master 稳定版、develop 开发主干）和临时分支（feature 功能分支、release 预发布分支、hotfix 紧急修补分支）。

41. Системы автоматической сборки: предпосылки появления

自动构建系统：出现的背景与必要性。

Возникли для автоматизации рутинных задач: компиляция, сборка ресурсов, запуск тестов, упаковка artifacts, устранение ошибок человеческого фактора.

为解决人工编译部署效率低、易出错等问题而诞生。自动完成编译、依赖管理、自动化测试、打包发布等一系列重复性工作。

42. Системы сборки: Make и Makefile.

构建系统：Make 与 Makefile。

Make — классический инструмент сборки на основе правил. Makefile содержит цели (target), зависимости (prerequisites) и команды (commands). Отслеживает время файлов.

Make 是基于目标和文件时间戳的传统构建工具。Makefile 定义了目标 (target)、依赖 (dependencies) 和构建指令 (commands)。

43. Системы сборки: Ant. Команды Ant.

构建系统：Apache Ant 及 XML 目标任务。

Ant — Java-ориентированный инструмент сборки. Использует build.xml. Основные элементы: `<target>` (цели), `<task>` (javac, jar, mkdir, delete).

Apache Ant 是基于 XML 的 Java 构建工具。核心配置文件 build.xml 包含项目 (project)、目标 (target) 和具体任务 (task, 如 javac, jar, copy)。

44. Системы сборки: Ant-ivy.

构建系统: Apache Ivy 依赖管理。

Apache Ivy — менеджер зависимостей для Ant. Автоматизирует скачивание jar-библиотек из репозитория (Maven Central) по файлу ivy.xml.

Apache Ivy 是专门为 Apache Ant 提供依赖管理的扩展工具。通过 ivy.xml 自动从远程 Maven 仓库下载和解析 Java Jar 包依赖。

45. Системы сборки: Maven. POM. Репозитории и зависимости.

构建系统: Maven、POM、仓库与依赖管理。

Maven основан на концепции POM (Project Object Model, pom.xml). Поддерживает транзитивные зависимости и централизованные репозитории (Local, Central, Remote).

Apache Maven 采用项目对象模型 (POM, pom.xml) 描述项目。自动管理传递性依赖, 并建立三级仓库体系 (本地仓库、中央仓库、远程私服)。

46. Maven: Структура проекта. GAV.

Maven: 标准项目目录结构与 GAV 标识。

Стандартная структура: src/main/java, src/test/java. Координаты GAV: GroupId (организация), ArtifactId (имя проекта), Version (версия).

Maven 约定优于配置的目录结构 (src/main/java 等); GAV 坐标唯一标识构件: GroupId (组织名)、ArtifactId (项目名)、Version (版本号)。

47. Maven: Зависимости. Жизненный цикл сборки. Плагины.

Maven: 依赖机制、生命周期与插件体系。

Жизненный цикл: validate -> compile -> test -> package -> verify -> install -> deploy. Вся работа выполняется плагинами (maven-compiler-plugin и др.).

构建生命周期包含 validate、compile、test、package、install 等阶段。所有的实际构建任务均由指定的 Maven 插件 (Plugins) 执行。

48. Системы сборки: Gradle. Преимущества и файл сборки.

构建系统: Gradle 的优势与构建脚本。

Gradle сочетает гибкость Ant и декларативность Maven. Использует DSL на Groovy/Kotlin (build.gradle). Преимущества: высокая скорость (кэширование, инкрементность).

Gradle 结合了 Ant 的灵活与 Maven 的规范。使用 Groovy/Kotlin DSL 编写 build.gradle。优势: 支持增量构建、构建缓存, 性能远高于 Maven。

49. Системы сборки: GNU autotools. Создание конфигурации проекта.

构建系统: GNU Autotools 项目配置与生成。

Инструменты (autoconf, automake). Разработчик пишет configure.ac и Makefile.am. Инструменты генерируют скрипт configure и итоговый Makefile.

C/C++ 项目构建工具链。编写 configure.ac 和 Makefile.am 模板，运行 autoconf/automake 生成可移植的 configure 脚本与 Makefile。

50. Системы сборки: GNU autotools. Конфигурация и сборка проекта.

GNU Autotools: 项目配置与编译步骤。

Классический процесс сборки для пользователя: `./configure` (проверка окружения и сборок), `make` (компиляция кода), `make install` (установка в систему).

终端用户通用构建三部曲: `./configure` (检测平台兼容性与生成 Makefile) -> `make` (编译生成二进制文件) -> `make install` (安装)。

51. Сервера сборки/непрерывной интеграции.

持续集成与自动化构建服务器 (CI/CD Server)。

CI-серверы (Jenkins, GitLab CI, GitHub Actions) автоматически запускают сборку и тесты при каждом коммите для раннего обнаружения ошибок.

CI 服务器（如 Jenkins、GitLab CI、GitHub Actions 等）在代码提交后自动触发编译与测试，确保及时发现并修复集成错误。

52. Основные понятия тестирования. Цели тестирования.

软件测试基本概念与主要目的。

Тестирование — процесс проверки соответствия ПО требованиям. Цели: выявление дефектов, проверка качества, снижение рисков, подтверждение готовности к релизу.

软件测试是验证软件是否符合预期需求的过程。主要目的：发现缺陷、评估质量、规避上线风险、提升用户满意度。

53. Понятие полного тестового покрытия и его достижимости. Пример.

全覆盖测试 (Full Coverage) 概念及其可实现性与示例。

Полное покрытие (испытание всех комбинаций входов и путей) практически невозможно из-за комбинаторного взрыва. Используются техники оптимизации (эквивалентность).

穷尽测试（测试所有输入组合与路径）因组合爆炸在现实中几乎不可行。因此需要采用等价类划分等技术进行测试集优化。

54. Статическое и динамическое тестирование.

静态测试与动态测试。

Статическое: проверка без запуска кода (ревью кода, статическое анализ, проверка ТЗ).

Динамическое: выполнение кода на тестовых наборах данных.

静态测试：不运行代码（代码审查、文档评审、静态静态静态分析）；动态测试：实际运行程序输入测试数据并验证输出结果。

55. Автоматизация тестов и ручное тестирование.

自动化测试与手动测试对比。

Ручное: выполняет человек (хорошо для UX, исследований). Автоматизированное: выполняют скрипты (для регрессии, нагрузочного, модульного тестирования).

手动测试由测试人员人工操作（适合探索性测试、UI/UX 评估）；自动化测试借助脚本运行（适合单元测试、回归测试、性能测试）。

56. Источники данных для тестирования. Роли и деятельности в тестировании.

测试数据来源及测试角色与活动。

Источники: ТЗ, спецификации, продуктовые логи, генераторы данных. Роли: QA Engineer, Test Lead, Automation Engineer. Деятельности: планирование, дизайн, прогон, отчет.

测试数据源：需求文档、用户日志、数据生成器；测试角色：QA 工程师、测试经理、自动化测试专家；测试活动：计划、设计、执行、缺陷跟踪。

57. Понятие тестового случая и сценария.

测试用例（Test Case）与测试场景（Test Scenario）。

Test Case: ID, предусловия, шаги, входные данные, ожидаемый результат. Test Scenario: последовательность несколько чеклистов/кейсов для проверки бизнес-фичи.

测试用例（Test Case）：包含编号、前置条件、操作步骤、预期结果；测试场景（Test Scenario）：验证某项特定业务功能的高层级业务流程组合。

58. Выбор тестового покрытия и количества тестов. Анализ эквивалентности.

测试覆盖度与测试量选择：等价类划分与边界值分析。

Разбиение входных данных на классы эквивалентности (Equivalence Partitioning) и выбор представителя из каждого класса + анализ граничных значений (Boundary Value).

等价类划分法：将输入域划分为若干等价类，每个类抽取代表值进行测试；边界值分析法：重点测试边界点及其左右侧相邻值。

59. Модульное тестирование. Junit 4 с примерами.

单元测试（Unit Testing）及JUnit 4 实践示例。

Тестирование отдельных функций/методов. JUnit 4 аннотации: `@Test`, `@Before`, `@After`, `@BeforeClass`, `@AfterClass`. Утверждения: `assertEquals()`, `assertTrue()`.

对软件最小可测试单元（函数、类）进行隔离验证。JUnit 4 常用注解：`@Test`、`@Before`、`@After` 及断言方法如 `assertEquals()`。

60. Интеграционное тестирование. Стратегии интеграции.

集成测试与常见集成策略。

Проверка взаимодействия между модулями. Стратегии: Восходящая (Bottom-Up), Нисходящая (Top-Down), «Большой взрыв» (Big Bang), Sandwich.

验证不同模块组合协作时的接口与交互。策略：自底向上（Bottom-Up）、自顶向下（Top-Down）、一次性集成（Big Bang）和夹层集成（Sandwich）。

61. Функциональное и End-to-End тестирование. Selenium.

功能测试、端到端测试 (E2E) 与 Selenium 工具。

E2E проверяет работу системы целиком от лица пользователя. Selenium — фреймворк для автоматизации действий пользователя в веб-браузерах.

E2E 测试从前端到后端验证完整的端到端业务流。Selenium 是最通用的 Web 浏览器自动化测试框架，模拟真实用户点击与输入。

62. Техники статического тестирования. Статический анализ кода.

静态测试技术与静态代码分析。

Анализ без запуска: Code Review, Walkthrough, Инспекции. Инструменты статанализа (SonarQube, SpotBugs) находят потенциальные баги, дубликаты, уязвимости.

静态分析不运行代码。技术包括同行评审（Code Review）、代码走查；静态分析工具（如 SonarQube）自动扫描代码漏洞、规范与重复度。

63. Тестирование системы в целом. Системное тестирование. Тестирование производительности.

系统级测试 (System Testing) 与性能测试。

Системное тестирование проверяет готовую систему в целом на соответствие всем требованиям.

Тестирование производительности проверяет скорость и устойчивость.

系统测试对完整的集成系统进行全方位检验（包含功能与非功能）。性能测试用于测量系统在给定负载下的响应速度与吞吐能力。

64. Тестирование системы в целом. Альфа- и бета-тестирование. Приемочное тестирование.

Alpha / Beta 测试与用户验收测试 (UAT)。

Альфа-тестирование (внутри компании разработчика). Бета-тестирование (конечными пользователями на реальных данных). UAT — финальная приемка заказчиком.

Alpha 测试由内部团队在开发环境进行；Beta 测试发布给部分最终用户在真实环境中体验；UAT（验收测试）由客户最终确认系统交付。

65. Аспекты быстродействия системы. Влияние средств измерения на результаты.

系统性能维度及测量工具对结果的影响 (Observer Effect).

Быстродействие зависит от CPU, памяти, I/O, сети. Средства измерения (профилировщики, метрики) создают накладные расходы (Overhead), искажая измерения.

性能与 CPU、内存、磁盘 I/O 及网络紧密相关。探针与分析工具自身会消耗资源（观察者效应/Overhead），造成测量结果的一定偏差。

66. Ключевые характеристики производительности.

性能测试的核心关键指标。

1) Response Time (время отклика), 2) Throughput (TPS/RPS - пропускная способность), 3) Resource Utilization (CPU, RAM, Disk I/O, Network), 4) Latency (задержка).

1) 响应时间 (Response Time); 2) 吞吐量 (Throughput/TPS); 3) 资源利用率 (CPU, Memory, I/O); 4) 延迟 (Latency) 与并发量。

67. Нисходящий метод поиска узких мест.

自顶向下 (Top-Down) 性能瓶颈排查法。

Анализ производительности от высокого уровня к низкому: Приложение -> Операционная система -> Hardware / Сеть. Позволяет быстро локализовать проблему.

从宏观到微观的性能诊断路径：首先排查应用层逻辑，其次分析 OS 系统指标（CPU/IO/内存），最后排查底层硬件与网络环境。

68. Пирамида памяти и ее влияние на производительность.

存储器金字塔 (Memory Hierarchy) 及其对性能的影响。

Иерархия: Регистры CPU -> L1/L2/L3 Cache -> RAM -> SSD/HDD. Чем выше, тем быстрее и дороже.

Пропахи кэша (Cache Miss) существенно снижают скорость.

存储层级：寄存器 -> L1/L2/L3 缓存 -> 主存 RAM -> 磁盘 SSD/HDD。速度递减而容量递增。Cache Miss（缓存未命中）是引发性能下降的主因之一。

69. Мониторинг производительности: процессы.

性能监控：进程级别监控 (Processes).

Отслеживание потребления ресурсов конкретными процессами (CPU %, RSS/VSZ RAM, Threads, FD).

Инструменты: `top`, `htop`, `ps`, `pidstat`.

监控特定进程的资源开销（CPU占用率、物理/虚拟内存使用、线程数、文件描述符）。常用工具：`top`, `htop`, `ps`, `pidstat`。

70. Мониторинг производительности: виртуальная память.

性能监控：虚拟内存与 Swapping 监控。

Анализ подкачки страниц (Paging/Swapping). Активный swap (si/so в `vmstat`) означает нехватку RAM и резко падает производительность системы.

监控内存分页与 Swap 交换区使用。如果出现频繁的 Swap 换入换出（`vmstat` 中的 si/so），意味着物理内存不足，会导致系统严重卡顿。

71. Мониторинг производительности: буферизированный файловый ввод-вывод.

性能监控：缓冲文件 I/O 监控。

Операции ввода-вывода (I/O). Страничный кэш (Page Cache) ускоряет чтение/запись. Инструменты `iostat`, `iotop` показывают tps, await, %util диска.

操作系统利用 Page Cache 缓冲文件读写。通过 `iostat`, `iotop` 监控磁盘每秒读写次数 (TPS)、等待时间 (await) 及磁盘利用率 (%util)。

72. Мониторинг производительности: Windows и Linux.

性能监控：Windows 与 Linux 工具对比。

Windows: Task Manager, Performance Monitor (PerfMon), Resource Monitor. Linux: `top`, `vmstat`, `iostat`, `sar`, `uptime`, `netstat`/`ss`.

Windows 工具：任务管理器、性能监视器 (PerfMon)、资源监视器；Linux 工具：`top`, `vmstat`, `iostat`, `sar`, `uptime`, `ss` 等。

73. Системный анализ Linux "за 60 секунд".

Linux 系统性能分析“60秒法则”。

Брендан Грегг предлагает 10 команд за 60 секунд: `uptime`, `dmesg | tail`, `vmstat 1`, `mpstat -P ALL 1`, `pidstat 1`, `iostat -xz 1`, `free -m`, `sar -n DEV 1`, `top`.

Brendan Gregg 提出的 60 秒排查命令集：`uptime`, `dmesg`, `vmstat 1`, `mpstat`, `pidstat 1`, `iostat -xz 1`, `free -m`, `sar`, `top`。

74. Создание тестовой нагрузки и нагрузчики.

测试负载生成与压力测试工具 (Load Generators)。

Имитация работы множества пользователей. Инструменты: Apache JMeter, Locust, k6, wrk, Gatling. Измеряют поведение системы при высокой нагрузке.

模拟大量并发用户向系统发起请求。常用工具：Apache JMeter, Locust, k6, wrk, Gatling。测量在高负载下系统的稳健性与响应。

75. Профилирование приложений. Основные подходы.

应用程序性能剖析 (Profiling) 与核心方法。

Анализ времени выполнения кода. Подходы: 1) Sampling (периодические снимки стека), 2) Instrumentation (внедрение кода замера в вызовы). Инструменты: Async-Profiler, JProfiler.

分析代码执行时间与资源占用。主要方法：1) 抽样法 (Sampling, 定期抓取堆栈)；2) 探针插桩法 (Instrumentation)。工具：Async-Profiler, JProfiler。

76. Компромиссы (trade-offs) в производительности.

性能优化中的权衡 (Trade-offs)。

Баланс между ресурсами: Память vs Время (кэширование увеличивает память, но ускоряет время), Сложность vs Скорость, Безопасность vs Быстродействие.

资源与效率间的权衡：空间换时间（缓存占内存但提速）、可读性 vs 极致性能、安全加密与计算开销之间的平衡。

77. Рецепты повышения производительности при высоком %SYS.

高 %SYS (系统 CPU 开销高) 时的性能优化方案。

Высокий %SYS означает частые системные вызовы. Решения: уменьшение системных вызовов (буферизация I/O, epoll), использование memory pools, оптимизация контекстных переключений. %SYS 高说明频繁触发内核态系统调用。优化方案：增加 I/O 缓冲减少 syscall 频次、使用 epoll 替代 poll/select、减少线程频繁切换。

78. Рецепты повышения производительности при высоком %IO wait.

%iowait (磁盘 I/O 等待高) 时的性能优化方案。

Высокий %iowait — узкое место в диске/сети. Решения: переход на SSD, кэширование в RAM (Redis/Memcached), асинхронный I/O, оптимизация SQL-запросов (индексы).

%iowait 高表示 CPU 在等待磁盘/网络 I/O。优化方案：使用 SSD 固态硬盘、引入 Redis 等内存缓存、采用异步 I/O、对数据库 SQL 加索引减少全表扫描。

79. Рецепты повышения производительности при высоком %Idle.

%Idle (CPU 空闲高但系统慢) 时的性能优化方案。

CPU свободен, но система тормозит. Причины: блокировки (locks), deadlocks, ожидание внешних API/БД. Решения: уменьшение синтаксических блокировок, асинхронные вызовы.

CPU 处于空闲状态但整体处理缓慢。原因：代码中存在死锁/锁竞争、线程阻塞等待外部 API 或数据库。优化方案：优化锁粒度、使用无锁结构及异步调用。

80. Рецепты повышения производительности при высоком %User.

高 %User (用户态 CPU 开销高) 时的性能优化方案。

Высокий %User означает интенсивные вычисления в коде приложения. Решения: оптимизация алгоритмов, устранение лишних циклов/объектов, профилирование кодовой базы.

%User 高说明业务应用程序自身在进行大量密集计算。优化方案：优化核心算法复杂度、减少不必要的对象创建与循环计算、通过 Profiler 寻找热点函数并重构。